AEGIS OS

Agents Component

Engineering Design Document (EDD)

Document ID	DOC-AEGIS-AGENTS-EDD-001
Version	1.5
Status	Draft
Date	March 2026
Component	Agents Component
Parent System	Aegis OS
Classification	Internal / Design

CONFIDENTIAL — Internal Design Document

1. Introduction

1.1 Purpose

This Engineering Design Document describes the technical architecture, module specifications, data flows, interfaces, and implementation guidance for the Agents Component of Aegis OS. It is intended for engineers building the component and for architects of adjacent components (Orchestrator, Memory, Credential Vault, Communications) who need to understand the integration contract.

1.2 Design References

- Aegis OS Official Specification v1.0 — technical stack, hypervisor, data layer
- Aegis OS PRD v1.5 — platform capabilities and foundational requirements
- Chroma Research: Context Rot (2025) — evidence base for the progressive disclosure constraint
- Anthropic: Progressive Disclosure of Agent Tools (2025) — CLI-style skill discovery pattern
- Together AI: How to Build a Coding Agent from Scratch (2025) — ReAct loop and tool contract patterns
- Nader Dabit / Mario Zechner: How to Build a Custom Agent Framework with PI (2026) — content/details split, compaction, session persistence

1.3 Key Design Decisions

Decision	Rationale
Progressive skill disclosure (CLI-style hierarchy)	Context rot research proves that pre-loading all capabilities degrades agent performance. Agents discover skills on demand, limiting context to what is actively needed.
Lazy credential delivery	Credentials are pre-authorized at spawn but not delivered until the agent explicitly requests them. Reduces exposure window and enforces least privilege.
Vault-executed operations (v1.5)	The Credential Vault executes credentialed operations on behalf of agents and returns results. Raw credential values never leave the Vault boundary. Agents submit execution requests; the Vault holds and uses the secret internally. See ADR-004.
Async transport for Vault operations, synchronous agent loop	Vault execute request/result uses separate durable NATS topics for idempotency and resilience. The agent loop remains semantically blocking per tool call. True parallel async dispatch is deferred to v2. See ADR-004.
Agent Registry backed by Memory Component	The Agents Component owns no persistent storage. Registry state is persisted via Memory Interface, enabling node-level statelessness and crash recovery.
Orchestrator as single integration point	The Agents Component communicates with exactly one external logical counterpart: the Orchestrator. All outbound events route through the Orchestrator, which coordinates with Credential Vault, Memory/Storage, and User I/O on the Agents Component's behalf.
Event-driven transport via Communications Component	All communication between the Agents Component and the Orchestrator is mediated exclusively through the Communications Component (NATS JetStream). No direct API calls, HTTP calls, or database connections.
Single internal Communications Interface gateway	Within the Agents Component, all inbound and outbound event bus interactions are funnelled through M1. No other internal module touches NATS directly.

Decision	Rationale
MicroVM per agent	Firecracker microVM isolation is the primary security boundary between agents. Compromise of one agent cannot affect another or the host OS.

2. System Context

2.1 Context Diagram

The Agents Component sits within the Aegis OS platform and communicates exclusively with the Orchestrator via the Communications Component (NATS JetStream). All inter-component coordination — credential operations, state persistence, user I/O — is mediated by the Orchestrator. The Agents Component has no direct relationship with any other component.

Note: Figure 1 (System Context Diagram) is maintained as a separate diagram artifact. See /docs/diagrams/system-context-v1.5.

2.2 External Interface Summary

Architectural constraint: The Agents Component has exactly one external logical counterpart — the Orchestrator. All communication flows to and from the Orchestrator via the Communications Component (NATS JetStream). The Agents Component has no direct relationship with Credential Vault, Memory/Storage, or User I/O. Those components are the Orchestrator's responsibility to coordinate.

External Component	Relationship Type	Transport	Notes
Orchestrator	DIRECT — sole external counterpart	NATS JetStream via Comms	Inbound: aegis.agents.* topics. Outbound: aegis.orchestrator.* topics. Full topic registry in §8.1.
Communications Component	INFRASTRUCTURE — transport only	NATS JetStream (is the bus)	Provisions streams and enforces access controls. Not a logical peer.
Credential Vault	INDIRECT — via Orchestrator	Orchestrator coordinates	Not consumed or published directly. Vault executes credentialed operations on behalf of agents (v1.5 change).
Memory/Storage	INDIRECT — via Orchestrator	Orchestrator coordinates	Not consumed or published directly.
User I/O	INDIRECT — via Orchestrator	Orchestrator coordinates	Not consumed or published directly.

3. Internal Architecture

3.1 Component Architecture Diagram

Note: Figure 2 (Internal Architecture Diagram) is maintained as a separate diagram artifact. See /docs/diagrams/internal-arch-v1.5.

3.2 Module Inventory

The Agents Component consists of seven modules. Each module has a single, well-defined responsibility. Modules communicate through defined internal interfaces, never directly manipulating each other's data.

M1: Communications Interface

Attribute	Detail
Responsibilities	Single inbound/outbound gateway to NATS JetStream. Handles envelope wrapping, deduplication on message_id, ACK/NACK logic, and inbound routing by message_type. Includes three Vault execution topics (request, progress, result) added in v1.5.
Inputs	Inbound Orchestrator events via aegis.agents.* topics; outbound publication requests from all internal modules.
Outputs	Parsed, validated events dispatched to internal modules; enveloped events published to aegis.orchestrator.* and aegis.orchestrator.vault.* topics.
Interfaces With	All internal modules (inbound routing + outbound publication), NATS JetStream (external).

M2: Agent Factory

Attribute	Detail
Responsibilities	Central coordinator for all agent provisioning decisions. On task receipt: queries Agent Registry for capability match. Routes to existing agent if match found; initiates provisioning if not. Orchestrates Skill Hierarchy Manager, Credential Broker, and Lifecycle Manager for new agent builds. Enforces spawn context budget of 2,048 tokens maximum.
Inputs	task_spec from Comms Interface; capability_result from Agent Registry; skill_entry_points from Skill Hierarchy Manager; permission_token from Credential Broker; vm_ready from Lifecycle Manager.
Outputs	query to Agent Registry; skill resolution request to Skill Hierarchy Manager; auth_request to Credential Broker; spawn instruction to Lifecycle Manager; publish task_result via Comms Interface.
Interfaces With	Comms Interface, Agent Registry, Skill Hierarchy Manager, Credential Broker, Lifecycle Manager.

M3: Agent Registry

Attribute	Detail
Responsibilities	Maintains the authoritative in-process catalog of all agents. Records per-agent: ID, vm_id, current state (7-state machine), skill domains, permission set token, health status, failure_count, state_history. Responds to capability queries. Persists all registry changes via Memory Interface.
Inputs	capability_query from Agent Factory; state_update from Lifecycle Manager; state.read.response via Comms Interface (on startup/recovery).
Outputs	capability_result to Agent Factory (match/partial_match/no_match); write request to Memory Interface.
Interfaces With	Agent Factory, Lifecycle Manager, Memory Interface.

M4: Skill Hierarchy Manager

Attribute	Detail
Responsibilities	Owns and serves the three-level skill tree: Domain → Command → Parameter Spec. Enforces progressive disclosure pattern. Supports both structural drill-down and semantic search (embedding-based). Rejects skill registrations that do not satisfy the Tool Contract (§13.2).
Inputs	skill_resolution_request from Agent Factory (at spawn); skills --help / drill-down queries from Agent MicroVM (at runtime).
Outputs	skill_entry_points to Agent Factory; domain list / command list / param spec to Agent MicroVM (one level at a time).
Interfaces With	Agent Factory (spawn time), Agent MicroVM (runtime), Memory Interface (persistence).

M5: Credential Broker

Attribute	Detail
Responsibilities	Pre-authorizes a scoped permission set for each agent at spawn. At runtime, validates individual agent credential requests against the pre-authorized scope before escalating to the Orchestrator. In v1.5, also routes Vault execution requests (aegis.orchestrator.vault.execute.request) for operations requiring credentialed access. Publishes revocation on agent termination. Logs every credential event as an audit write. Raw credential values never pass through this module.
Inputs	auth_request from Agent Factory (spawn); vault_execute_request from Agent MicroVM (runtime); aegis.agents.credential.response and aegis.agents.vault.execute.result from Comms Interface.
Outputs	aegis.orchestrator.credential.request (authorize/revoke); aegis.orchestrator.vault.execute.request (execute); audit write request to Memory Interface.
Interfaces With	Agent Factory, Agent MicroVM, Comms Interface, Memory Interface (audit).

M6: Lifecycle Manager

Attribute	Detail
Responsibilities	Sole authority for launching, suspending, resuming, and terminating agent microVMs. Monitors agent health via heartbeat. On failure: instructs Memory Interface to publish a state snapshot before attempting recovery. Decides restart vs. replace based on failure_count vs. max_retries. Manages the full 7-state machine (PENDING → SPAWNING → ACTIVE → IDLE → SUSPENDED → RECOVERING → TERMINATED).
Inputs	spawn_instruction from Agent Factory; activate/suspend/terminate instructions; heartbeat from Agent MicroVM.
Outputs	vm_ready status to Agent Factory; state_update to Agent Registry; snapshot write request to Memory Interface; revoke instruction to Credential Broker.
Interfaces With	Agent Factory, Agent Registry, Agent MicroVM, Memory Interface, Credential Broker.

M7: Memory Interface

Attribute	Detail
Responsibilities	Disciplined state persistence gateway. Packages write requests into structured aegis.orchestrator.state.write events. Packages read requests into aegis.orchestrator.state.read.request events. Enforces structured, tagged payloads — untagged payloads are rejected. Enforces minimal read filters (agent_id + data_type
Inputs	Tagged write payloads from Agent Registry, Credential Broker, Lifecycle Manager, Agent MicroVM; read requests from Agent Registry, Agent Factory.
Outputs	aegis.orchestrator.state.write events; aegis.orchestrator.state.read.request events; state.read.response delivered back to requesting module.
Interfaces With	Agent Registry, Credential Broker, Lifecycle Manager, Agent MicroVM, Comms Interface.

4. Data Flow Diagram

Note: Figure 3 (Data Flow Diagram) is maintained as a separate diagram artifact. See /docs/diagrams/data-flow-v1.5.

4.1 Primary Data Flows

Important: All flows that cross the Agents Component boundary travel through the NATS JetStream bus to the Orchestrator. The Orchestrator then coordinates downstream with Credential Vault, Memory/Storage, and User I/O as needed. Flows labelled 'internal' remain within the Agents Component process boundary and never touch the bus.

Flow 1: Task Reception (cross-component, via Orchestrator)

- The Orchestrator publishes a task_spec event to aegis.agents.task.inbound on the NATS bus.
- The Communications Interface (M1) consumes the event, validates the envelope, and dispatches the payload to Agent Factory.
- task_spec includes: task_id, required_skill_domains[], priority, timeout_seconds, payload, and optional user_context_id.
- Agent Factory immediately publishes aegis.orchestrator.task.accepted with agent_type and estimated_completion_at.

Flow 2: Registry Lookup (internal)

- Agent Factory queries Agent Registry with required_skill_domains[].
- Registry checks its in-memory catalog. If a match exists (IDLE or SUSPENDED agent), factory routes to the fast activation path.
- Registry returns match | partial_match | no_match with available_agents[] and missing_domains[].
- If no match, factory initiates the full provisioning flow.

Flow 3: Skill Resolution (internal)

- Agent Factory requests skill_entry_points for each required domain from Skill Hierarchy Manager.
- Manager serves only the top-level domain entry point per domain — nothing deeper. This is an internal call only.
- Agent Factory enforces the 2,048 token spawn context budget. Provisioning aborts if exceeded.

Flow 4: Credential Pre-Authorization (cross-component, via Orchestrator)

- Credential Broker publishes a credential.request event (operation: authorize) to aegis.orchestrator.credential.request via the Communications Interface.
- The Orchestrator receives this event and coordinates with the Credential Vault internally. The Agents Component is not involved in that interaction.
- The Orchestrator publishes the result — a scoped permission_token — back to aegis.agents.credential.response.
- Communications Interface delivers the response to Credential Broker. The agent receives only the permission_token reference, never a raw secret.

Flow 5: MicroVM Spawn (internal)

- Agent Factory instructs Lifecycle Manager to spawn a new agent with: task payload, skill_entry_points, permission_token.
- Lifecycle Manager transitions agent through PENDING → SPAWNING → ACTIVE states.
- On success, Agent Registry is updated. Registry instructs Memory Interface to persist the state change.

Flow 6: Progressive Skill Discovery (internal, agent runtime)

- The running agent issues skills --help to the Skill Hierarchy Manager. Entirely internal — does not cross the bus.
- Manager returns domain list. Agent drills down: skills <domain> --help → commands, skills <domain> <command> --help → full param spec.
- Only the level explicitly requested is returned. Full param specs are never pre-loaded.
- Semantic search (skills --search 'query') is also supported for ambiguous task domains. Returns top-3 command summaries only.

Flow 7: Vault-Executed Operations (cross-component, via Orchestrator) — NEW in v1.5

- When an agent invokes a skill requiring a credentialed operation, it submits a vault execute request to the Credential Broker (internal call).
- Credential Broker validates the request against the pre-authorized permission set. If valid, it publishes aegis.orchestrator.vault.execute.request with the operation parameters, permission_token, and agent-specified timeout_seconds.
- The Orchestrator routes the request to the Credential Vault. The Vault retrieves the credential internally, executes the operation, and publishes progress events to aegis.agents.vault.execute.progress during long-running operations.
- On completion, the Vault publishes the operation result to aegis.agents.vault.execute.result. The result carries the operation output — never the raw credential value.
- The agent's Act phase is blocked (goroutine yielded) awaiting the result. The local deadline is set to timeout_seconds + 5s buffer. If the deadline fires first, the tool returns TOOL_TIMEOUT and a cancellation is published.
- Raw credential values never leave the Vault boundary at any point in this flow.

Flow 8: State Persistence (cross-component, via Orchestrator)

- On task completion or checkpoint, the agent produces a tagged payload. Memory Interface publishes a state.write event to aegis.orchestrator.state.write.
- The Orchestrator coordinates with the Memory/Storage Component internally and publishes a write acknowledgement to aegis.agents.state.write.ack.
- Memory Interface confirms the ack before signalling completion to the requesting module.

Flow 9: Result Delivery (cross-component, via Orchestrator)

- Agent Factory instructs Communications Interface to publish a task_result event to aegis.orchestrator.task.result.
- The Orchestrator consumes the result and distributes it as needed. The Agents Component considers the task closed on publication.

5. Sequence Diagrams

5.1 New Agent Provisioning — Full Flow

This diagram covers the complete provisioning sequence from task receipt through to a running agent executing its first skill query. This is the most complex path and the one the Agents Component is primarily designed to execute well.

Note: Figure 4 (New Agent Provisioning Sequence Diagram) is maintained as a separate diagram artifact at /docs/diagrams/seq-provisioning-v1.5. Key change from v1.4: the credential.response step no longer carries a raw credential value to the agent. Instead, the agent submits vault execute requests and receives operation results.

5.2 Existing Agent Assignment — Short Path

When the Agent Registry finds a capable IDLE or SUSPENDED agent, the provisioning flow is bypassed entirely. No skill resolution, no credential pre-authorization, no microVM spawn. The existing agent is activated and assigned directly. This is the fast path and should be the most common flow in a mature deployment.

Note: Figure 5 (Existing Agent Assignment Sequence Diagram) is maintained as a separate diagram artifact at /docs/diagrams/seq-existing-agent-v1.5.

5.3 Vault-Executed Operation — Async Flow (NEW in v1.5)

This diagram covers the async request/result flow for a single Vault-executed tool call within a running agent's ReAct loop.

Note: Figure 6 (Vault Execute Sequence Diagram) is maintained as a separate diagram artifact at /docs/diagrams/seq-vault-execute-v1.5.

6. Agent Lifecycle

6.1 Lifecycle State Machine

Note: Figure 7 (Agent Lifecycle State Machine) is maintained as a separate diagram artifact at /docs/diagrams/lifecycle-state-machine-v1.5.

6.2 State Definitions

State	Description	Entry Condition	Valid Transitions
PENDING	Agent has been requested but not yet created.	No capable agent found in registry; factory initiates provisioning.	→ SPAWNING (provisioning starts) → TERMINATED (provisioning fails immediately)
SPAWNING	MicroVM is being configured and launched.	Lifecycle Manager receives spawn instruction.	→ ACTIVE (vm_ready received) → TERMINATED (launch fails after retries)
ACTIVE	Agent is running and executing a task.	MicroVM ready and task assigned.	→ IDLE (task complete) → RECOVERING (crash detected)
IDLE	Agent has completed its task; microVM running but unassigned.	Task completion confirmed.	→ ACTIVE (new task assigned) → SUSPENDED (idle timeout) → TERMINATED (explicit decommission)
SUSPENDED	Agent state preserved; microVM paused or stopped to free resources.	Idle timeout exceeded.	→ ACTIVE (wake-up triggered by new task; Credential Broker issues fresh token) → TERMINATED (explicit decommission)
RECOVERING	Agent has crashed; Lifecycle Manager attempting recovery.	Heartbeat failure or crash signal detected.	→ ACTIVE (recovery successful; respawned with same agent_id, new vm_id) → TERMINATED (max retries exceeded)
TERMINATED	Agent has been permanently removed from service.	Max retries exceeded, explicit decommission, or clean shutdown.	None (terminal state). Registry record archived.

6.3 Recovery Behavior

When a crash is detected (heartbeat timeout or explicit crash signal), the Lifecycle Manager executes the following recovery sequence:

1. Save last known state: Memory Interface is called to persist the most recent agent state checkpoint via aegis.orchestrator.state.write.
2. Inspect in-flight Vault operations: the session log is checked for any vault execute requests that were in-flight at the time of crash (identified by request_id with no corresponding result). These are flagged as UNKNOWN — the Vault may or may not have executed them.

3. Determine recovery strategy: if failure_count < max_retries, attempt microVM respawn with recovered state. If failure_count >= max_retries, transition to TERMINATED.
4. Respawn: Lifecycle Manager launches a fresh microVM with the same task context and recovered state. The agent_id is preserved; only vm_id changes. This is transparent to the Orchestrator.
5. Credential re-authorization: Credential Broker publishes a fresh credential.request (operation: authorize) for the recovered agent. The previous token may have expired during the crash interval.
6. Resume from checkpoint: the agent reads its session log back from Memory via state.read.request, determines which tool calls completed successfully, and decides whether to resubmit in-flight Vault operations using the same request_id (Vault deduplicates on request_id — safe to resubmit).
7. Registry update: Agent Registry is updated with new vm_id but same agent_id, and failure_count is incremented. On successful task completion, failure_count is reset to 0.

Design note: Recovery is 'same agent, new VM'. The agent's identity (agent_id, task context, state) persists through recovery. Only the microVM is replaced. In-flight Vault operations that cannot be confirmed as completed must be resubmitted with their original request_id — the Vault's idempotency guarantee ensures they execute exactly once.

7. Data Models

7.1 Agent Registry Record Schema

Stored in Memory Component via Memory Interface. One record per agent.

Field	Type	Description
agent_id	UUID (string)	Globally unique agent identifier. Persists through recovery and microVM replacement.
vm_id	string | null	Current Firecracker microVM instance ID. Changes on respawn. Null if SUSPENDED or TERMINATED.
state	enum	Current lifecycle state: PENDING | SPAWNING | ACTIVE | IDLE | SUSPENDED | RECOVERING | TERMINATED
skill_domains	string[]	List of skill domain names this agent is provisioned for (e.g., ['web', 'data']).
permission_set_token	string	Reference token for pre-authorized credential scope. Never a raw secret.
task_id	UUID | null	Currently assigned task ID. Null if IDLE or SUSPENDED.
created_at	ISO 8601	When the agent was first created.
last_active_at	ISO 8601	Last time the agent was in ACTIVE state.
failure_count	integer	Number of consecutive failures since last successful task. Reset to 0 on success. Compared against max_retries in recovery logic.
state_history	StateEvent[]	Ordered list of state transitions: { state, timestamp, reason }. Append-only.

7.2 Task Specification Schema

Received from the Orchestrator via the Communications Interface.

Field	Type	Required	Description
task_id	UUID	YES	Unique identifier. Idempotency key — duplicate task_ids rejected after first receipt.
required_skill_domains	string[]	YES	Skill domains required to execute this task (e.g., ['web', 'storage']).
priority	integer (1-10)	YES	Task execution priority. Higher values = higher priority for concurrent provisioning.
timeout_seconds	integer	YES	Maximum allowed execution time before the task is considered failed.
payload	object	YES	Task-specific data payload. Opaque to the Agents Component; passed directly to the agent.
user_context_id	UUID | null	NO	Optional User I/O session reference. Echoed back in all outbound events for this task.

7.3 Skill Hierarchy Schema

Level	Fields	Example
Domain (Level 1)	domain_id, name, description, command_count	{ domain_id: 'web', name: 'Web', description: 'HTTP operations', command_count: 5 }
Command (Level 2)	command_id, domain_id, name, description, required_credential_types[]	{ command_id: 'fetch', domain_id: 'web', description: 'Fetch a URL', required_credential_types: ['web_api_key'] }
Param Spec (Level 3)	command_id, parameters: {name, type, required, description, default}[]	{ command_id: 'fetch', parameters: [{name: 'url', type: 'string', required: true}] }

7.4 Memory Interface Write Payload Schema

All writes to the Memory Component must conform to this structure. Untagged payloads are rejected.

Field	Type	Description
agent_id	UUID	Source agent identifier.
data_type	enum	One of: agent_state | task_result | skill_cache | audit_log | credential_event | episode | snapshot | capability_profile | execution_pattern
timestamp	ISO 8601	Time the data was generated.
payload	object	Structured payload specific to the data_type. Raw text/ transcripts are rejected.
ttl_seconds	integer | null	Optional time-to-live. After expiry, Memory Component may garbage-collect. Null = retain indefinitely.
require_ack	boolean	If true, Agents Component waits for state.write.ack before proceeding. Default true for snapshots and audit events.

8. Interface Specifications

The Agents Component has a single external logical interface: the Orchestrator. Every event we publish goes to an aegis.orchestrator.* topic. Every event we consume arrives on an aegis.agents.* topic published by the Orchestrator. v1.5 adds three Vault execution topics.

8.1 Master Topic Registry

Topic	Publisher	Consumer	Delivery	Persisten	Purpose
aegis.agents.task.inbound	Orchestrator	Agents	At-least-once	24h	Task assignment — new work for the Agents Component.
aegis.agents.capability.query	Orchestrator	Agents	At-most-once	Transient	Query: do we have capable agents for these skill domains?
aegis.agents.lifecycle.terminate	Orchestrator	Agents	At-least-once	24h	Instruction to terminate a specific agent immediately.
aegis.agents.credential.response	Orchestrator	Agents	At-least-once	1h	Credential operation result (authorize/revoke) from Vault via
aegis.agents.vault.execute.progress	Orchestrator	Agents	At-most-once	Transient	NEW v1.5: Progress heartbeat events from the Vault during long-running operations. Informational only — losing one
aegis.agents.vault.execute.result	Orchestrator	Agents	At-least-once	1h	NEW v1.5: Final result of a Vault-executed operation. Carries operation output, never
aegis.agents.state.write.ack	Orchestrator	Agents	At-least-once	24h	Acknowledgement that a state.write was persisted by Memory/Storage via
aegis.agents.state.read.response	Orchestrator	Agents	At-least-once	1h	State records returned by Memory/Storage in response to our read request.
aegis.agents.clarification.response	Orchestrator	Agents	At-least-once	1h	User I/O response to a clarification question, routed
aegis.orchestrator.task.accepted	Agents	Orchestrator	At-least-once	24h	Confirm task received and agent assigned; includes agent_type and estimated_completion_at.
aegis.orchestrator.task.result	Agents	Orchestrato	At-least-once	7 days	Final task output on completion.
aegis.orchestrator.task.failed	Agents	Orchestrator	At-least-once	7 days	Task failure with error_code, phase, retryable flag, and
aegis.orchestrator.capability.response	Agents	Orchestrator	At-most-once	Transient	Response to capability query: match/partial_match/no_match with agent details.
aegis.orchestrator.agent.status	Agents	Orchestrato	At-least-once	24h	Agent state transition
aegis.orchestrator.credential.request	Agents	Orchestrator	At-least-once	1h	Request Orchestrator to coordinate a credential operation (authorize/revoke) with the Vault.

Topic	Publisher	Consumer	Delivery	Persisten	Purpose
aegis.orchestrator.vault.execute.request	Agents	Orchestrator	At-least-once	1h	NEW v1.5: Request Orchestrator to execute a credentialed operation via the Vault. Carries operation parameters and agent-specified timeout_seconds. Vault
aegis.orchestrator.state.write	Agents	Orchestrator	At-least-once	24h	Request Orchestrator to persist a structured state payload to Memory/Storage.
aegis.orchestrator.state.read.request	Agents	Orchestrator	At-least-once	1h	Request Orchestrator to retrieve state records from Memory/
aegis.orchestrator.clarification.request	Agents	Orchestrator	At-least-once	1h	Request Orchestrator to surface a question to the user via User I/
aegis.orchestrator.audit.event	Agents	Orchestrator	At-least-once	Permanent	Security and audit events for persistence and monitoring (append-only).
aegis.orchestrator.error	Agents	Orchestrator	At-least-once	72h	Component-level errors requiring Orchestrator awareness.

8.2 Message Envelope Standard

Every event published by the Agents Component must include the standard envelope. The Communications Component uses the envelope for routing and deduplication. Unwrapped payloads are rejected.

Field	Type	Required	Description
message_id	UUID	YES	Globally unique message ID. Used by the Communications Component for deduplication within the idempotency window.
message_type	string	YES	Dot-notation type matching the topic purpose. E.g., task.result, credential.request, vault.execute.request.
source_component	string	YES	Always 'agents'. Used for routing and audit correlation.
correlation_id	UUID	YES	The task_id, query_id, or request_id this message is responding to or originating from.
timestamp	ISO 8601	YES	Event generation time at source (not bus receipt time).
schema_version	string	YES	Semver string, e.g. '1.0'. Enables graceful schema evolution.
payload	object	YES	The event-specific payload. Schema defined per topic below.

8.3 Task Flow Event Schemas

aegis.agents.task.inbound — Payload: TaskSpec (inbound)

Field	Type	Required	Constraints	Description
task_id	UUID	YES	Globally unique	Idempotency key. Duplicate task_ids rejected after first receipt.
required_skill_domains	string[]	YES	Min 1 item	Skill domains the executing agent must possess.

Field	Type	Required	Constraints	Description
priority	integer	YES	1–10	Higher = higher priority for concurrent provisioning.
timeout_seconds	integer	YES	30–86400	Hard deadline. Agent terminated and task.failed returned if exceeded.
payload	object	YES	Max 1MB serialised	Task data. Opaque to Agents Component. Passed verbatim to agent.
user_context_id	UUID | null	NO	Nullable	Optional User I/O session reference. Echoed in all outbound events.

aegis.orchestrator.task.accepted — Payload (outbound)

Field	Type	Description
task_id	UUID	Echoed from the originating TaskSpec.
agent_id	UUID	The agent assigned to this task (existing or newly provisioned).
agent_type	string	new_provision | existing_assigned — informs Orchestrator of provisioning cost incurred.
estimated_completion_at	ISO 8601 | null	Best-effort estimate based on task timeout and current load. Null if not computable.
user_context_id	UUID | null	Echoed from TaskSpec.

aegis.orchestrator.task.result — Payload (outbound)

Field	Type	Description
task_id	UUID	Echoed from the originating TaskSpec.
agent_id	UUID	The agent that executed the task.
status	enum	success | timed_out | cancelled
payload	object	Task output. Verbatim result from agent execution. Opaque to Agents Component.
user_context_id	UUID | null	Echoed for Orchestrator to route output to correct User I/O session.
execution_ms	integer	Wall-clock task execution duration in milliseconds.
user_message	string | null	Human-readable summary. Never contains technical error detail or stack traces.

aegis.orchestrator.task.failed — Payload (outbound)

Field	Type	Description
task_id	UUID	Echoed from the originating TaskSpec.
agent_id	UUID | null	The agent involved. Null if failure occurred before assignment.
error_code	string	Machine-readable code. E.g., PROVISIONING_FAILED, TIMEOUT, SCOPE_VIOLATION, RECOVERY_EXHAUSTED, VAULT_UNREACHABLE, CLARIFICATION_TIMEOUT.
phase	string	Where failure occurred: registry_lookup | skill_resolution | credential_auth | vm_spawn | execution | recovery | vault_execute.
retryable	boolean	If true, Orchestrator may resubmit with a new task_id.
user_message	string	Human-readable failure description suitable for user display.
user_context_id	UUID | null	Echoed for Orchestrator routing.

8.4 Capability Query Schemas

aegis.agents.capability.query — Payload (inbound)

Field	Type	Required	Description
query_id	UUID	YES	Idempotency key for this query.
required_skill_domains	string[]	YES	Domains to check availability for.
availability_required	enum	NO	any_state | idle_only | active_capable. Default: idle_only.

aegis.orchestrator.capability.response — Payload (outbound)

Field	Type	Description
query_id	UUID	Echoed from the CapabilityQuery for correlation.
result	enum	match | partial_match | no_match
available_agents	AgentSummary[]	Agents matching the query: { agent_id, state, skill_domains, last_active_at }. Empty on no_match.
missing_domains	string[]	Domains from the query no current agent covers. Empty if result = match.
provisioning_estimate_seconds	integer | null	Estimated time to provision a new agent. Null if not computable.

8.5 Credential Operation Schemas

v1.5 architectural change: The Credential Vault now executes credentialed operations on behalf of agents and returns operation results. Raw credential values never leave the Vault boundary. The 'fetch' operation type has been removed and replaced by the new Vault execute request/result topic pair. Credential operations via this topic (authorize, revoke) remain for lifecycle management only.

aegis.orchestrator.credential.request — Payload (outbound)

Field	Type	Required	Description
request_id	UUID	YES	Correlation key — echoed in the credential.response.
operation	enum	YES	authorize | revoke — credential lifecycle operations only. ('fetch' removed in v1.5 — use vault.execute.request instead.)
agent_id	UUID	YES	The agent this operation applies to.
task_id	UUID	YES	The task this agent is executing. Used for audit correlation.
skill_domains	string[]	YES (authorize)	Required on authorize. Orchestrator uses this to derive vault policy scope.
ttl_seconds	integer	YES (authorize)	Required on authorize. Requested token lifetime = task timeout + 300s buffer.
permission_token	string	YES (revoke)	Required on revoke. The opaque token issued on authorize.

aegis.agents.credential.response — Payload (inbound)

Field	Type	Description
request_id	UUID	Echoed from the credential.request for correlation.
operation	enum	authorize | revoke — echoed from request.
agent_id	UUID	Echoed for verification.
status	enum	granted | denied | revoked | error
permission_token	string | null	Present on successful authorize only. Opaque reference token.
expires_at	ISO 8601 | null	Token expiry. Present on authorize success.
error_reason	string | null	Present if status = denied or error. Must not contain vault internals.

aegis.orchestrator.vault.execute.request — Payload (outbound) — NEW v1.5

Field	Type	Required	Description
request_id	UUID	YES	Idempotency key. Vault deduplicates on this field — safe to resubmit after crash.
agent_id	UUID	YES	The agent submitting this operation.
task_id	UUID	YES	Associated task. Used for audit correlation.
permission_token	string	YES	The opaque token from the prior authorize operation. Used by Vault to validate scope.
operation_type	string	YES	The type of credentialed operation to execute (e.g., web_fetch, storage_read). Must match a registered operation type in the Vault.
operation_params	object	YES	Typed parameters for the operation. Schema defined per operation_type by the Vault.

Field	Type	Required	Description
timeout_seconds	integer	YES	Agent-specified maximum execution time. The Vault enforces this as its hard deadline. Range: 1–300.
credential_type	string	YES	The credential type required (e.g., web_api_key). Vault uses this to resolve the correct secret internally.

aegis.agents.vault.execute.progress — Payload (inbound) — NEW v1.5

Field	Type	Description
request_id	UUID	Echoed from the vault.execute.request for correlation.
agent_id	UUID	Echoed for verification.
progress_type	enum	heartbeat | milestone — heartbeat is time-based; milestone is a semantic event (e.g., connection_established, response_headers_received).
message	string | null	Human-readable progress description. E.g., 'Response headers received, reading body (42KB)'
elapsed_ms	integer	Milliseconds elapsed since the operation started.
timestamp	ISO 8601	When this progress event was emitted.

Note: Progress events are at-most-once transient. Losing a progress event does not affect operation correctness. The Orchestrator (and subscribed monitoring consumers) may use progress heartbeats to detect stalled Vault operations independently of the agent's timeout.

aegis.agents.vault.execute.result — Payload (inbound) — NEW v1.5

Field	Type	Description
request_id	UUID	Echoed from the vault.execute.request for correlation.
agent_id	UUID	Echoed for verification.
status	enum	success | timed_out | scope_violation | execution_error
operation_result	object | null	The output of the executed operation. Schema defined per operation_type. Present on success only. NEVER contains raw credential values.
error_code	string | null	Machine-readable error code. Present if status != success.
error_message	string | null	Human-readable error description. Must not contain credential internals or vault path details.
elapsed_ms	integer	Total execution time in milliseconds.
timestamp	ISO 8601	When the Vault completed the operation.

Security invariant: The operation_result field carries operation output only — never a raw credential value. The credential used to execute the operation is retrieved and used entirely within the Vault boundary and is never transmitted to the Agents Component at any point.

8.6 State Persistence Schemas

aegis.orchestrator.state.write — Payload (outbound)

Field	Type	Required	Description
request_id	UUID	YES	Correlation key — echoed in state.write.ack.
agent_id	UUID	YES	Agent this state record belongs to.
data_type	enum	YES	agent_state | task_result | episode | snapshot | skill_cache | audit_log | credential_event | capability_profile | execution_pattern
payload	object	YES	Structured payload for this data_type. Raw text and transcripts are rejected.
ttl_seconds	integer | null	NO	Optional TTL. Null = retain indefinitely.
require_ack	boolean	NO	If true, Agents Component waits for state.write.ack before proceeding. Default true for snapshots and audit events.

8.7 User Clarification Schemas

aegis.orchestrator.clarification.request — Payload (outbound)

Field	Type	Required	Description
request_id	UUID	YES	Correlation key.
task_id	UUID	YES	The task this clarification is blocking.
user_context_id	UUID	YES	The User I/O session to route the question to.
question	string	YES	The question to present to the user. Plain text, max 500 chars.
options	string[] | null	NO	Optional structured choices for UI rendering as a selection widget.
timeout_seconds	integer	YES	If no response received within this window, fail task with CLARIFICATION_TIMEOUT.

8.8 Status and Audit Events

aegis.orchestrator.audit.event — Payload

Field	Type	Description
event_type	enum	credential_grant | credential_deny | credential_revoke | scope_violation | vault_execute_request | vault_execute_result | vault_execute_timeout | state_transition | provisioning_start | provisioning_complete | provisioning_fail | recovery_attempt | task_accepted | task_completed | task_failed
agent_id	UUID	Agent involved in the event.
task_id	UUID | null	Associated task, if applicable.

Field	Type	Description
request_id	UUID | null	For vault_execute_* events: the request_id of the Vault execute operation.
initiating_module	string	Which internal module generated this event (e.g., CredentialBroker, LifecycleManager).
outcome	enum	success | denied | failed | partial
detail	object	Structured detail specific to event_type. No raw text, no PII, no credential values, no operation results.
timestamp	ISO 8601	Immutable event time. Orchestrator must store this append-only.

8.9 Internal Interfaces (Intra-Component Only)

Agent MicroVM → Skill Hierarchy Manager

This interaction is internal. It does not cross the event bus.

- skills --help → returns: { domains: [{domain_id, name, description, command_count}] }
- skills <domain> --help → returns: { domain_id, commands: [{command_id, name, description}] }
- skills <domain> <command> --help → returns: { command_id, parameters: [...], required_credential_types: [...] }
- skills --search '<query>' → returns: top-3 matching commands with domain paths (name + description only, no parameters).

No other query form is supported. An agent cannot retrieve more than one level of skill detail per structural query. This constraint is enforced by the Skill Hierarchy Manager.

Agent MicroVM → Credential Broker (Vault Execute)

This interaction is internal. The Credential Broker translates the request into an aegis.orchestrator.vault.execute.request event — the agent never touches the bus directly.

- Input: { permission_token, operation_type, operation_params, timeout_seconds, credential_type }
- Broker validates scope against the pre-authorized permission set before escalating. Scope violation → SCOPE_VIOLATION error returned internally + audit event emitted. No Orchestrator event published.
- On escalation: Broker publishes vault.execute.request and yields the calling goroutine.
- Progress events (from aegis.agents.vault.execute.progress) are forwarded to the tool's onUpdate callback for monitoring. They do not enter LLM context.
- On result receipt: Broker returns { operation_result, elapsed_ms } to the agent. No credential value is ever present in this return.
- On timeout: Broker returns { error: 'TOOL_TIMEOUT' } and publishes a cancellation to the Orchestrator.

9. Security Design

9.1 Agent Isolation

- Each agent runs in an independent Firecracker microVM. No shared memory, no shared process space.
- MicroVM destruction is the security boundary: a terminated or failed agent leaves no executable state on the host.
- The host OS (Alpine Linux, diskless mode) is immutable and read-only. No agent can modify host state.

9.2 Credential Security Model

v1.5 change: Raw credential values now never leave the Vault boundary at any point. The previous model delivered credential values to agents in memory; the new model has the Vault execute operations internally using credentials it holds, returning only operation results.

- Pre-authorization (spawn time): Credential Broker publishes a credential.request (operation: authorize) to the Orchestrator. The Orchestrator coordinates with the Credential Vault and returns a scoped permission_token. No raw secret crosses any boundary at this stage.
- Vault-executed operations (runtime): when an agent invokes a skill requiring a credentialed operation, the Credential Broker validates the request against the pre-authorized scope, then publishes a vault.execute.request to the Orchestrator. The Orchestrator routes this to the Vault. The Vault retrieves the credential internally, executes the operation, and returns only the operation result. The credential value is never transmitted outside the Vault.
- Token TTL: all permission tokens are time-limited. A SUSPENDED agent's token expires during suspension; Credential Broker publishes a fresh authorize request on wake-up.
- Revocation on termination: when Lifecycle Manager transitions an agent to TERMINATED, Credential Broker publishes a credential.request (operation: revoke) to the Orchestrator. The Orchestrator revokes the token in the Vault.
- Scope violation handling: out-of-scope vault execute requests are denied internally by the Credential Broker before any Orchestrator event is published. A scope_violation audit event is emitted immediately.
- Bus-level security: all events are protected by mTLS at the transport layer. The aegis.agents.vault.execute.result topic must only be accessible to the Agents Component's consumer group — no other component may subscribe.
- Operation result security: the operation_result field in vault.execute.result carries operation output only. Audit events for vault_execute_* must never include the operation_result payload — only metadata (request_id, operation_type, status, elapsed_ms).

9.3 Audit Trail

- Every credential event (authorize, revoke, scope_violation) results in an audit.event published to aegis.orchestrator.audit.event.
- Every Vault execute operation (request, result, timeout, scope_violation) produces an audit record. The record includes request_id, operation_type, status, and elapsed_ms. It never includes operation_params, operation_result, or any credential-adjacent detail.
- Every state transition produces an audit record via Memory Interface → Orchestrator → Memory/Storage.
- Audit records are append-only. The Orchestrator must never allow these to be modified or deleted.

10. Error Handling and Resilience

10.1 Provisioning Failures

Failure Scenario	Detection	Response
Skill resolution fails (unknown domain)	Skill Hierarchy Manager returns internal error	Factory publishes aegis.orchestrator.task.failed with phase=skill_resolution. No agent created.
Credential authorize request gets no response	No aegis.agents.credential.response within timeout	Credential Broker retries up to 3 times with exponential backoff. On persistent failure, factory publishes task.failed with error_code VAULT_UNREACHABLE.
Orchestrator returns credential status=denied	Credential Broker receives denied credential.response	Factory publishes task.failed with error_code VAULT_AUTH_DENIED. Audit event published.
MicroVM launch fails	Lifecycle Manager receives error from Firecracker	Retry on a different node if available. On persistent failure, mark TERMINATED and publish task.failed.
State write gets no acknowledgement	No aegis.agents.state.write.ack within timeout	Retry publish up to 3 times. On persistent failure, abort spawn, destroy MicroVM, publish task.failed. No orphaned agents.
Spawn context budget exceeded	Agent Factory token count > 2,048 at provisioning time	Provisioning aborts. Factory publishes task.failed with error_code CONTEXT_BUDGET_EXCEEDED, phase=skill_resolution.

10.2 Runtime Failures

Failure Scenario	Detection	Response
Agent heartbeat timeout	Lifecycle Manager misses N consecutive heartbeats	Publish state.write for snapshot. Attempt recovery (see §6.3). Publish aegis.orchestrator.agent.status with new_state=RECOVERING.
Agent requests out-of-scope vault operation	Credential Broker internal scope validation fails	Deny internally (no Orchestrator event published). Emit scope_violation audit event. Optionally terminate agent (policy-configurable).
Vault execute request times out	Agent's local deadline (timeout_seconds + 5s buffer) fires before result received	Tool returns TOOL_TIMEOUT content to LLM. Cancellation published to Orchestrator. Vault audit event emitted. Agent may retry if task logic allows.
Vault execute progress events stop before timeout	Orchestrator detects stalled operation independently via progress heartbeat gap	Orchestrator may cancel the operation and publish a vault.execute.result with status=execution_error. Agents Component receives this result on the result topic
State write.ack returns status=rejected	Memory Interface receives rejected ack	Log rejection_reason. Do NOT silently discard. Retry with corrected payload if applicable. Publish error event if unresolvable.
Bus event goes undelivered (NATS NAK or timeout)	Comms Interface receives NAK or no ACK within timeout	At-least-once topics: republish after backoff. Dead-letter after max_redelivery. Vault execute topics: resubmit with same request_id for idempotent re-execution.

Failure Scenario	Detection	Response
Vault execute result never arrives	Agent's local deadline fires	Tool returns TOOL_TIMEOUT. If agent crashes before deadline, recovery flow resubmits with original request_id (Vault deduplicates).

11. Open Questions and Future Considerations

ID	Question	Impact	Owner
OQ-01	What is the exact retry and backoff policy for agent recovery? Should it be globally configured or per-agent-type?	Affects resilience SLA and resource usage during failure events.	Agents Component team + Platform team
OQ-02	How is the Skill Hierarchy tree updated at runtime? Who has write access and what is the invalidation strategy for cached sub-trees?	Critical for keeping agents up-to-date with platform capabilities without requiring restarts.	Agents Component team
OQ-03	What is the SUSPENDED-to-ACTIVE wake-up latency target? Does it need to be as fast as IDLE-to-ACTIVE?	Determines whether SUSPENDED is a viable state for latency-sensitive use cases.	Platform team + Orchestrator team
OQ-04	What are the Orchestrator's latency SLAs for credential.response and state.write.ack? These directly affect the Agents Component's provisioning time NFR.	Critical: without defined SLAs from the Orchestrator, the Agents Component cannot guarantee its own provisioning time targets.	Orchestrator team + Agents Component team
OQ-05	RESOLVED (see §13.3): Initial spawn context budget is defined as 2,048 tokens maximum. Runtime compaction triggers at 80% of the model's context window. Both are enforced by the Agents Component.	Resolved.	Agents Component team
OQ-06	Should agents that successfully complete tasks be preferentially reused, and if so, for how long should they remain IDLE before SUSPENSION?	Tuning this balance affects provisioning cost vs. latency.	Platform team
OQ-07	What compaction strategy produces the most accurate summary for agent recovery? Summarise-and-evict vs. selective retention?	Poor compaction summaries cause agents to lose task context.	Agents Component team + LLM quality team
OQ-08	True mid-task agent steering (redirecting a running agent without termination) requires a streaming steering event delivered to the running microVM. Deferred to v2.	Requires the ReAct loop to become a proper async state machine. Shared requirement with OQ-09.	Agents Component team
OQ-09	True parallel async tool dispatch (agent continues while Vault operates) requires the ReAct loop to become an async state machine. Deferred to v2.	Content/details split, compaction, and session persistence all become significantly more complex. Decision: Option A (async transport, synchronous loop) adopted for v1. See ADR-004.	Agents Component team

ID	Question	Impact	Owner
OQ-10	NEW: What is the progress event emission contract for Vault-executed operations? Time-based heartbeat interval, semantic milestones, or both? Should the interval be specified by the agent in the execute request or configured at the Vault level per operation_type?	Determines stall detection sensitivity and monitoring granularity. A time-based heartbeat is required for correctness; semantic milestones are a quality-of-life improvement. The interval per operation_type affects how quickly the Orchestrator can detect a stalled Vault execution.	Agents Component team + Vault team

13. Agent Runtime Specification

This section defines the internal execution model of a running agent. It specifies what happens inside an agent during task execution: the runtime loop, the tool contract, context budget management, session persistence, and workflow composition patterns. This is the authoritative runtime specification — an agent that does not implement it is not a valid Aegis OS agent.

13.1 The Agent Execution Loop (ReAct Pattern)

All Aegis OS agents implement a ReAct-style execution loop: a structured cycle of reasoning, tool invocation, result observation, and context updating that repeats until the task is complete or a termination condition is met.

Phase	Name	What Happens	Termination Condition
1	Reason	The agent LLM receives the current context window (task, system prompt, history, current tool results) and produces one of: a text response (task complete), a tool call request, or a clarification request.	Text response with no tool call → task complete. Token budget exceeded → trigger compaction before next phase.
2	Act (Tool Call)	The agent runtime dispatches the tool call to the appropriate skill. If the skill requires a credentialed operation: (a) the Credential Broker validates scope, (b) publishes aegis.orchestrator.vault.execute.request with operation parameters and timeout_seconds, (c) the calling goroutine yields awaiting the result topic. If the skill does not require a credential, execution proceeds locally within the microVM.	Tool execution error → log error, return error content to LLM, continue loop. timeout_seconds + 5s buffer fires before result received → return TOOL_TIMEOUT content, publish cancellation, continue loop. Max retries exceeded → transition to task failure.
3	Observe	The tool result is received. For Vault-executed operations, the result is the operation_result from vault.execute.result — never a raw credential. The result is split into content (returned to LLM, enters context) and details (emitted as monitoring event via Orchestrator, never enters agent context). Content is appended to conversation history.	Result exceeds max content size (16KB) → truncate and append truncation notice before injecting into context.
4	Update Context	The conversation history grows by one turn (tool call + result). The runtime checks the current token count against the compaction threshold. If threshold exceeded, compaction runs before the next Reason phase.	Token count < 80% of model context window → no action. Token count >= 80% → trigger §13.3 Compaction before next Reason phase.

13.2 Tool Contract — Required Schema for Every Skill

Every skill defined in the Skill Hierarchy must satisfy the following contract. The Skill Hierarchy Manager enforces this contract; skills that do not conform are rejected at registration time.

Field	Type	Required	Purpose	Notes
name	string	YES	Identifier used by the LLM in its tool call request.	Must be unique within the agent's active skill set. Snake_case. Max 64 chars.

Field	Type	Required	Purpose	Notes
label	string	YES	Human-readable display name for monitoring and UI.	Shown in agent.status events and audit logs. Never shown to the LLM.
description	string	YES	Tells the LLM what this tool does, when to use it, and when NOT to use it.	This is the highest-leverage field. Max 300 chars. Must include negative guidance.
parameters	JSON Schema object	YES	Typed parameter specification. Validated before execution.	Must define all parameters with descriptions. Parameters without descriptions cause LLM hallucination.
required_credential_types	string[]	CONDITIONAL	Declares which credential types this tool needs. v1.5: this is an execution routing declaration — it tells the Credential Broker which operations must be proxied through the Vault executor. Also used at spawn time to derive pre-authorization scope.	Required if and only if the tool accesses external services. Must exactly match credential types defined in the Skill Hierarchy.
execute	async function	YES	Implements the tool logic. For credentialed operations: submits vault execute request to Credential Broker and awaits result. For uncredentialed operations: executes locally within the microVM.	Must return: { content: MessageContent[], details: object }. Content enters LLM context. Details do not. onUpdate callback is for forwarding Vault progress events to monitoring.
timeout_seconds	integer	NO	Maximum execution time. For Vault-executed operations, this is passed directly as the timeout in the vault.execute.request.	Default: 30s. Hard max: 300s. If exceeded, tool returns error content: TOOL_TIMEOUT.

Tool Result Structure

Field	Type	LLM Sees It?	Max Size	Guidelines
content	MessageContent[]	YES	16KB	What the LLM needs to know to continue the task. For Vault-executed operations: the operation_result from vault.execute.result, summarized to relevant fields. Never include raw response bodies, credential hints, or vault path details.
details	object	NO	No limit	Full operational detail for monitoring: raw response bodies, timing, byte counts, full error traces, Vault elapsed_ms. Published as a monitoring event to the Orchestrator. Never injected into agent context.

13.3 Context Budget and Compaction Strategy

Token Budget Thresholds

Threshold	Value	Enforcement	Rationale
Spawn context budget	2,048 tokens maximum	Enforced by Agent Factory at provisioning time. Provisioning aborts if the initial context (system prompt + task + skill entry points + permission token) exceeds this limit.	Prevents context rot before the first tool call. Forces lean provisioning. Skill descriptions at spawn time are domain-level only — no command or parameter detail.
Compaction threshold	80% of model context window	Enforced by agent runtime before each Reason phase. If current token count >= threshold, compaction runs before the LLM is called.	Preserves 20% headroom for the current turn's output. Prevents hitting the hard context window limit mid-task.
Hard abort	95% of model context window	If token count reaches 95% (e.g., compaction failed), the agent runtime aborts the current turn and emits a CONTEXT_OVERFLOW error via the Orchestrator.	Safety net to prevent unpredictable LLM behaviour at the very edge of the context window.

Compaction Algorithm

When the compaction threshold is triggered, the runtime executes the following summarise-and-evict strategy:

1. Identify the compaction window: all turns older than the most recent N turns, where N = 10 (configurable). The most recent N turns are always retained verbatim.
2. Summarise: the runtime issues an internal LLM call with the compaction window as input and a structured summarisation prompt. The prompt instructs the summariser to: preserve all tool call outcomes, preserve all intermediate task state, preserve any commitments or constraints established in earlier turns, and discard conversational filler, redundant observations, and raw content that was already processed.
3. Validate the summary: the summary is token-counted. If the summary is larger than 25% of the compaction window's original size, the summarisation failed. In this case, a simpler extractive summary is attempted: retain only tool call names, status codes, and key extracted values.
4. Replace: the compaction window turns are removed from context and replaced with a single structured summary turn, marked with role=system and prefixed with [COMPACTED SUMMARY — turns N through M].
5. Persist: the compaction event is written to the agent's episodic memory (episode_type=compaction_event) via the Orchestrator, including the original turn range and the summary text.

13.4 Session Persistence Model

Concept	Description	Implementation
Session entry	One record per turn: user message, assistant response, or tool call/result pair.	Written as a structured JSON record to the agent's episodic memory via state.write events. Each entry has: entry_id (UUID), parent_entry_id (UUID | null), turn_type, content, timestamp.

Concept	Description	Implementation
Vault execute tracking	In-flight vault execute requests are recorded in the session log by request_id before the goroutine yields.	On recovery, the agent inspects the session log for request_ids with no corresponding result entry. These are resubmitted with the same request_id — the Vault deduplicates.
Append-only writes	Entries are never modified. New information always creates a new entry.	Enforced by the Orchestrator's audit_log append-only rule extended to session entries.
Tree structure	Each entry points to its parent via parent_entry_id. Enables branching for recovery.	Branch point: when recovery restarts an agent, a new entry is created pointing to the last known-good parent. The failed branch is retained for audit.
Compaction entry	A special entry type that replaces a range of prior entries with a summary.	entry_type=compaction. Contains: summary_text, first_compacted_entry_id, last_compacted_entry_id.
Recovery read	On agent recovery, the runtime reads back the most recent session branch to reconstruct context.	Via state.read.request to the Orchestrator, filtered to is_latest=true snapshots plus uncompacted turn entries since the last snapshot.

13.5 Skill Retrieval — Semantic vs. Structural

Mode	When Used	How It Works	Context Cost
Structural (default)	Agent knows the domain it needs (e.g. 'web', 'data').	Agent calls skills --help → gets domain list. Drills down to specific command. Three round trips maximum to get full param spec.	Very low. Only the requested level is returned.
Semantic (search)	Agent needs a capability but doesn't know which domain it's in.	Agent calls skills --search 'send notification' → Skill Hierarchy Manager performs semantic similarity search across all skill descriptions using pre-computed embeddings. Returns top-3 matching commands with their domain paths.	Low-medium. Returns 3 command summaries (name + description only, no parameters).

13.6 Workflow Composition Patterns

Pattern	Description	Agents Component Role
Sequential	Agent A completes task, then Agent B receives A's output as its task input.	Fully supported. task.result from A becomes the payload in the next task.inbound to B.
Parallel	Multiple agents receive the same (or different) tasks simultaneously. Results aggregated by Orchestrator.	Fully supported. Multiple task.inbound events result in multiple independent agent assignments.

Pattern	Description	Agents Component Role
Conditional	The Orchestrator evaluates an agent's result and routes to different next agents based on outcome.	Supported via the retryable flag in task.failed and the status field in task.result.
Iterative (Steering)	The Orchestrator needs to redirect a running agent mid-task without terminating it.	Partially supported: the Orchestrator can send a task_cancel followed by a new task with context from the original task. True mid-task steer without termination is deferred to v2 (see OQ-08).
Agent-as-tool	An agent invokes another agent as a sub-task.	Not supported in v1. All agent spawning is initiated by the Orchestrator. Tracked as future capability.

13.7 Research References

Source	Title	Key Contributions to This Design
Together AI, 2025	How to Build a Coding Agent from Scratch	Foundational tool-calling pattern; confirmed selective context retrieval as essential; defined the generate→execute→observe→iterate execution loop; documented four workflow composition patterns.
Nader Dabit / Mario Zechner, 2026	How to Build a Custom Agent Framework with PI	Introduced the content/details split in tool results; demonstrated context compaction (summarise-and-evict); defined the steer vs. followUp distinction; established JSONL append-only tree-structured session persistence.
Chroma Research, 2025	Context Window Performance Degradation Study	Provided the empirical basis for context rot as a design constraint; established the requirement for distractor-aware context management — the foundation of both progressive disclosure and compaction.

14. Architecture Decision Records

Architecture Decision Records document significant architectural choices — the decision taken, the alternatives considered, and the rationale. They exist so that future team members, auditors, and partner teams can understand why the system is shaped the way it is.

ADR-001 — Native Go Runtime Over PI Framework

Attribute	Detail
Status	ACCEPTED
Date	February 25, 2026
Deciders	Agents Component Team

The PI agent framework was identified as a strong implementation reference. The question was whether to adopt PI as a direct dependency for the agent runtime running inside Firecracker microVMs.

Decision: implement the agent runtime natively in Go. Do not take a runtime dependency on PI or any Node.js agent framework. Use PI as a specification reference only — its design patterns are captured in EDD §13 and credited in §13.7.

Concern	Detail
Architectural mismatch	PI is designed as a self-contained agent runtime that owns the loop, tool execution, and session persistence end-to-end. Aegis OS has a fundamentally different architecture: all state persistence flows through NATS events to the Orchestrator (not local disk), and all external calls are gated through the Credential Broker. Integrating PI would require overriding its I/O layer, its persistence layer, and its tool execution path simultaneously — effectively maintaining a significant fork of an upstream project.
Runtime footprint	Alpine Linux on Firecracker starts in ~125ms with ~20MB baseline memory. A Node.js runtime adds ~40-60MB per VM and increases cold start time by ~300-500ms. Given that fast provisioning is a top-tier NFR, this footprint cost is not acceptable. A Go agent runtime compiles to a small static binary with sub-millisecond startup overhead.
Single-maintainer risk	PI is maintained by a single developer. Depending on it as a core infrastructure component for an OS-level agent platform introduces supply chain concentration risk.
The adapter problem	Integrating PI requires: (1) a NATS adapter for all session persistence writes, (2) a Credential Broker adapter for all tool-level credential requests, (3) a custom compaction hook, (4) a microVM scoping adapter. At this point, PI is providing only the ReAct loop and TypeBox schema validation — both of which are straightforward to implement natively.

ADR-002 — Orchestrator-as-Hub Over Peer-to-Peer Event Topology

Attribute	Detail
Status	ACCEPTED
Date	February 25, 2026
Deciders	Agents Component Team + Orchestrator Team

The initial v1.2 architecture had the Agents Component publishing directly to all five peer components via their own topic namespaces. This is a peer-to-peer event topology where the Agents Component has five integration surfaces to maintain.

Decision: collapse all inter-component communication to a single peer — the Orchestrator. The Agents Component publishes exclusively to aegis.orchestrator.* and consumes exclusively from aegis.agents.*. The Orchestrator is responsible for all downstream coordination.

Concern	Detail
Integration surface reduction	Five bilateral contracts become one. Testing, mocking, and fault isolation all become dramatically simpler — stub the Orchestrator and the entire component is testable in isolation.
Responsibility clarity	In the peer-to-peer model, the Agents Component had to understand Memory write semantics, Vault policy naming conventions, and User I/O session routing — concerns that belong to other components.
Auditability	All Agents Component output flows through one choke point. The Orchestrator can log, rate-limit, and validate every outbound event without the Agents Component having to instrument each peer connection separately.
Tradeoff acknowledged	The Orchestrator becomes a more complex component. This is preferable to distributing that complexity across all five components.

ADR-003 — Anthropic Go SDK Directly, No Agent Framework Layer

Attribute	Detail
Status	ACCEPTED
Date	February 25, 2026
Deciders	Agents Component Team

The Agents Component implements the agent runtime (ReAct loop, tool calling, context management, session persistence) using the Anthropic Go SDK (github.com/anthropics/anthropic-sdk-go) directly. No agent framework layer is added between the Agents Component and the SDK. The EDD §13 serves as the authoritative runtime specification that guides the native implementation.

ADR-004 — Async Transport for Vault-Executed Operations, Synchronous Agent Loop

Attribute	Detail
Status	ACCEPTED
Date	March 2026
Deciders	Agents Component Team

Context: the v1.5 architectural change moves credential operations from a raw-value delivery model to a Vault-execution model. The question is whether Vault execute operations should use a synchronous or asynchronous communication pattern, and whether the agent loop should block waiting for Vault results or continue other work.

Decision: design the Vault execute transport as async (separate durable NATS topics correlated by request_id) while keeping the agent loop semantically blocking per tool call.

Option	Description	Outcome
Option A: Async transport, synchronous loop (ACCEPTED)	Vault execute request and result use separate durable NATS topics. The agent's Act phase publishes the request, yields the calling goroutine, and resumes when the result topic delivers the response. The loop is sequential from the agent's perspective.	ACCEPTED — see rationale below.
Option B: True async — agent continues while Vault operates	The agent loop becomes an async state machine capable of handling multiple in-flight tool calls simultaneously.	REJECTED — deferred to v2. Requires fundamental redesign of the ReAct loop, compaction logic, and session persistence model. Shared with OQ-08 (mid-task steering).

Rationale	Detail
Idempotency	The Vault deduplicates on request_id. An agent that crashes during a Vault operation can safely resubmit on recovery without risk of double-execution.
Resilience	Separate topics mean no held connections. The result arrives whenever the Vault is ready. Timeout handling is clean: the agent subscribes with a deadline derived from timeout_seconds + 5s buffer. If the deadline fires, the tool returns TOOL_TIMEOUT and publishes a cancellation — no ambiguity about whether the operation ran.
Observability	The Orchestrator can see all in-flight Vault execute requests independently of their results. Progress events (at-most-once transient) give the Orchestrator and monitoring consumers independent visibility into long-running operations without holding open correlation state.
Loop simplicity	The ReAct loop in §13.1 does not change structurally. The Act phase's goroutine yield is a natural Go concurrency primitive. No async state machine required in v1.
Progress events	The agent specifies timeout_seconds per operation in the execute request. The Vault publishes progress events (time-based heartbeats and semantic milestones) to aegis.agents.vault.execute.progress during long-running operations. Progress events are forwarded to the tool's onUpdate callback for monitoring without entering LLM context.

This decision is revisited if: (a) parallel async tool dispatch becomes a product requirement; (b) the ReAct loop needs to handle multiple concurrent tool calls for performance reasons; or (c) mid-task steering (OQ-08) is promoted to a v1 requirement, as both share the same underlying async state machine requirement.

Document Revision History

Version	Date	Author	Description
1.0	February 21, 2026	Design Team	Initial EDD. Architecture, 7 core modules, provisioning model, lifecycle, memory paradigms, security.
1.1–1.2	February 21, 2026	Design Team	Event-driven architecture: all interactions via NATS JetStream.
1.3	February 25, 2026	Design Team	Orchestrator-as-hub revision. Single external peer, simplified topic namespace.
1.4	February 25, 2026	Design Team	Added §13 (Agent Runtime Specification) and §14 (ADR-001, ADR-002). Added ADR-003 (Anthropic Go SDK).
1.5	March 2026	Design Team	Vault architectural change: Vault executes credentialed operations and returns results; raw credential values never leave Vault boundary. Added ADR-004 (async transport, synchronous loop). New vault execute topic trio (request, progress, result). Updated §8.5 (removed fetch operation, added execute schemas). Updated §9.2 security model. Updated §13.1 Act phase. Updated §13.2 tool contract. Updated §6.2 state definitions (7 states). Updated §6.3 recovery (in-flight Vault operation handling). Updated §7.1 AgentRecord (vm_id, failure_count, state_history). Updated §8.3 task schemas (task.accepted, task.failed). Updated §8.4 capability response (match/partial_match/no_match). Updated §8.8 audit events (vault execute event types). Added OQ-10 (progress event emission contract). Added §13.2 vault execute tracking to session persistence model.